# Circuit Switchers: Direct-Mapped Cache Controller

Modern Computer Architecture - Project 6

Guru Manohar Gupta, Harsith Priyan, Pranavi Mathur
Yatin Singh, Dadi Dinesh
Department of CS and AI
Newton School of Technology
Phase I – 14 November 2026

***Abstract*—Main-memory access can limit processor performance. Circuit Switchers implements a direct-mapped read-only cache in Verilog with 32-bit byte addresses, 64 lines and 16-byte blocks, for 1 KiB of data. A four-state controller returns one byte per request and refills the indexed line on a miss. Self-checking Icarus Verilog runs verified 331 completed reads and three reset cancellations using an independent residency model and deterministic memory contents. With the default simulated memory latency, hits and misses took two and eight clock cycles, respectively. Average memory access time ranged from 2.375 to 8.000 cycles across four workload traces. Synthesis evaluated the cache logic with an external memory interface.**

***Index Terms*—direct-mapped cache, Verilog, memory hierarchy, RTL verification**

## I. Introduction

Caches retain recently used blocks and can reduce the time spent waiting for memory. The Circuit Switchers prototype handles one outstanding read, returns an 8-bit response and refills with aligned 128-bit blocks. The delivered RTL baseline is Git commit `9a227c8`. This milestone covers the integrated read path and synchronous active-high reset.

## II. Literature Review

Smith surveys cache placement, block size and trace-driven evaluation [1]. Temporal locality motivates retaining recently accessed data. Spatial locality motivates fetching adjacent bytes in a block. The repeated and sequential traces exercise these effects. Hennessy and Patterson describe quantitative evaluation of the memory hierarchy, including the effect of cache misses on average access time [2].

Direct mapping assigns each memory block to exactly one cache line [1], [3]. An index selects the line, a tag identifies the resident block, and a valid bit indicates whether it can be used. Blocks sharing an index can repeatedly evict one another.

## III. Proposed Solution

### A. Configuration and address fields

The implemented geometry is summarized in Table I. The data capacity is 64 x 16 = 1024 bytes. Six index bits select 64 lines, four offset bits select 16 bytes and the remaining 22 bits form the tag. Metadata adds 64 x (22 + 1) = 1472 bits, or 184 bytes; the advertised 1 KB refers to data capacity alone.

Fig. 1 gives an overview of the integrated read path. Fig. 2 shows the exact assignments in `rtl/address_decoder.v`: tag = address[31:10], index = address[9:4], and offset = address[3:0]. For example, 0x00000000 and 0x00000400 both select line 0, with tags 0 and 1 respectively. The alternating-address workload confirms replacement at this index.

TABLE I
Implemented Cache Configuration

| Property | Value |
|---|---|
| Address | 32-bit byte address |
| Data capacity | 1024 bytes (1 KB) |
| Lines / block | 64 / 16 bytes (128 bits) |
| Tag / index / offset | 22 / 6 / 4 bits |
| Data / tag / valid storage | 8192 / 1408 / 64 bits |
| Processor response | 8-bit read with valid and hit |

### B. Implemented storage and reset

`cache_data_ram` stores 64 128-bit blocks and `tag_ram` stores 64 22-bit tags. Both expose independent read and write indexes, combinational reads, and synchronous writes gated by `write_en`. They have no reset ports; before initialization, their values may be unknown in simulation.

`valid_bit_ram` stores one bit per line. Active-high synchronous reset clears all 64 bits and takes priority over a write. An enabled write can set or clear one bit. The refill path ties `write_valid` to one. Lookup requires a valid line and a matching 22-bit tag.

### C. Top-level connections and transaction flow

`cache_top` decodes `req_addr` and passes the fields to the controller. The controller captures them when both `req_valid` and `req_ready` are sampled high. These fields drive reads, tag comparison and byte selection (Fig. 3). The tag comparator sits inside `cache_lookup`. Separate selectors read the cached and refill blocks. Lane $n$ occupies bits $8n$ through $8n + 7$. Appending four zero bits to the captured tag and index forms the aligned memory address.

Figs. 4 and 5 show the implemented sequence. IDLE accepts a request and LOOKUP examines the indexed line. On a hit, the controller latches the cached byte and enters RESPOND with the hit flag asserted. On a miss, MEM_WAIT holds `mem_req` and the aligned address until `mem_ready`. Ready enables the data, tag and valid writes. On the refill edge, the controller captures the requested byte directly from the returned block. RESPOND drives a one-cycle `resp_valid`

TABLE II
OBSERVED WORKLOAD RESULTS

| Run | Reads | Hits | Misses | Hit (%) | Cycles/read |
|---|---|---|---|---|---|
| Directed | 11 | 3 | 8 | 27.27 | 6.364 |
| Sequential | 32 | 30 | 2 | 93.75 | 2.375 |
| Repeated | 16 | 15 | 1 | 93.75 | 2.375 |
| Conflict | 16 | 0 | 16 | 0.00 | 8.000 |
| Random | 256 | 39 | 217 | 15.23 | 7.086 |

pulse. Synchronous reset restores IDLE, invalidates every line and cancels any in-flight memory request.

The 64 KiB simulation model aliases addresses modulo 65,536. Default contents at byte address $a$ are the XOR of address bits [7:0], bits [15:8] and `0xA5`. The model asserts ready four periods after its own acceptance edge.

## IV. VERIFICATION AND RESULTS

### A. Regression and scoreboard

The decoder, storage, lookup and controller tests passed 6, 22, 41 and 40 checks, respectively. The controller test exercises the integrated cache. Real-only top-level elaboration also passed. Five additional simulations model block residency independently and check address capture, response byte, hit flag, latency and single completion per uncanceled request. On misses the scoreboard verifies alignment, the handshake, all refill lanes and the installed line. Hits must leave memory idle.

The directed run covered cold fill, two resident indexes, backpressure, full-width tags, warm-cache invalidation and reset cancellation during lookup, memory wait and memory ready. The sequential trace covers all 16 offsets and crosses a block boundary. The repeated trace reads `0x04` 16 times. The conflict trace alternates `0x00000000` and `0x00000400`. The 256-read random trace uses xorshift32 with seed `0x6D2B79F5` over 4 KiB. Each trace starts cold.

### B. Latency and workload measurements

Latency counts rising-edge intervals from request acceptance to consumer sampling of `resp_valid`. With acceptance at E0, a hit asserts valid after E1 and is sampled at E2. On a miss, memory accepts at E2 and asserts ready after E6. The cache refills at E7 and the requester samples the response at E8. The additional miss penalty is six cycles. For miss fraction $p_m$, AMAT is $2 + 6p_m$ cycles per read. It matches the direct mean of sampled latencies for each workload. The simulation clock period is 10 ns.

Table II counts completed reads. The rates exclude three reset-canceled requests. AMAT also excludes idle intervals. Figs. 7 and 8 show the cold refill, repeat hit and same-index replacement. Fig. 6 shows a separate controller regression.

### C. Generic synthesis and reproducibility

Yosys 0.71+34 synthesized the cache logic with the simulation memory excluded and the memory interface exposed. The flattened flow produced 32,190 generic cells including nine scope metadata cells, with 9,715 flip-flop cells and 3,499 multiplexer cells. The measurements cover generic mapping and RTL simulation. FPGA utilization, physical area, maximum frequency and speedup relative to a no-cache baseline were outside the measured scope. The repository includes commands, statistics, regression logs, per-read traces, source hashes and VCDs.

## V. INDIVIDUAL CONTRIBUTIONS

**Guru Manohar Gupta (Member 01)** implemented the address decoder and top-level integration, and supplied their interface, architecture and decoder checks (commits `9aa6577`, `d930302`, `1217060`, `370a8f7`).

**Harsith Priyan (Member 02)** implemented data, tag and valid storage and supplied directed verification and documentation (commits `5662211`, `f434789`, `114d788`, `d6e5ccc`).

**Pranavi Mathur (Member 03)** implemented tag comparison, validity-gated lookup and byte selection with directed checks and documentation (commits `3b44e0e`, `cb44829`).

**Yatin Singh (Member 04)** implemented the controller and memory model and supplied the integrated controller test, FSM and memory documentation, and waveform (commit `7cf0ef7`).

**Dadi Dinesh (Member 05)** delivered the full-system scoreboard and deterministic workloads, regression scripts, hit/miss and AMAT analysis, VCD evidence, generic synthesis flow, verification documentation and technical revision of the Phase I report.

## VI. CONCLUSION

The integrated read path passed directed and seeded workloads with byte-lane, reset and conflict checks. The two-cycle hit and eight-cycle miss were consistent across the runs at the default memory latency. Differences in miss fraction account for the observed AMAT range.

## REFERENCES


[1] A. J. Smith, Cache memories, *ACM Computing Surveys*, vol. 14, no. 3, pp. 473–530, Sep. 1982, doi: 10.1145/356887.356892.

[2] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017, ch. 2, ISBN 978-0-12-811905-1.

[3] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed. Morgan Kaufmann, 2020, ch. 5, ISBN 978-0-12-820331-6.

# APPENDIX A
# ARCHITECTURE AND WAVEFORM EVIDENCE

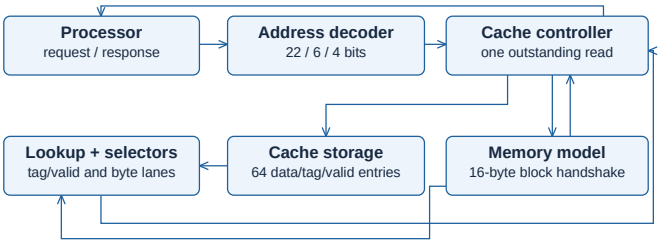



Fig. 1. Integrated read path with simulated memory.

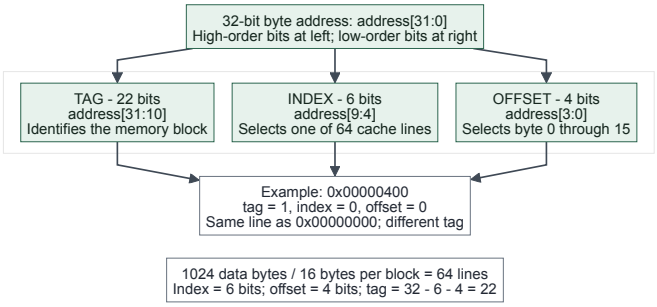



Fig. 2. Address fields and a worked example.

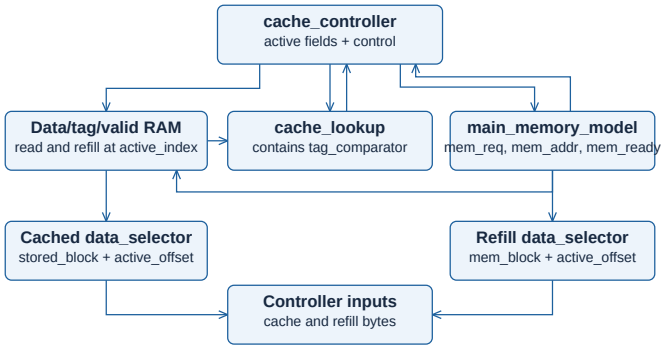



Fig. 3. Controller, storage, lookup and byte selection. The memory block feeds both the refill selector and the cache data RAM.

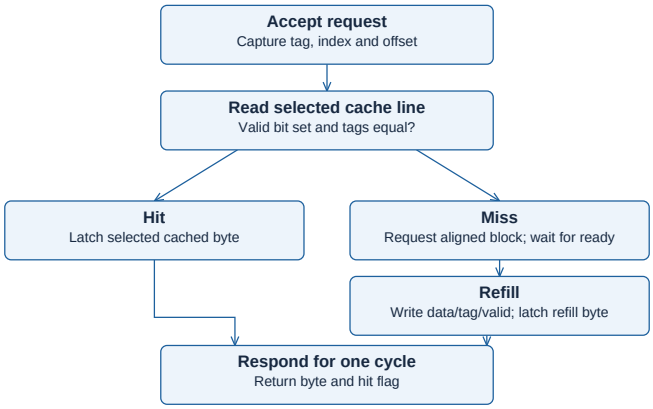



Fig. 4. The refill writes the indexed line and returns the selected memory byte.

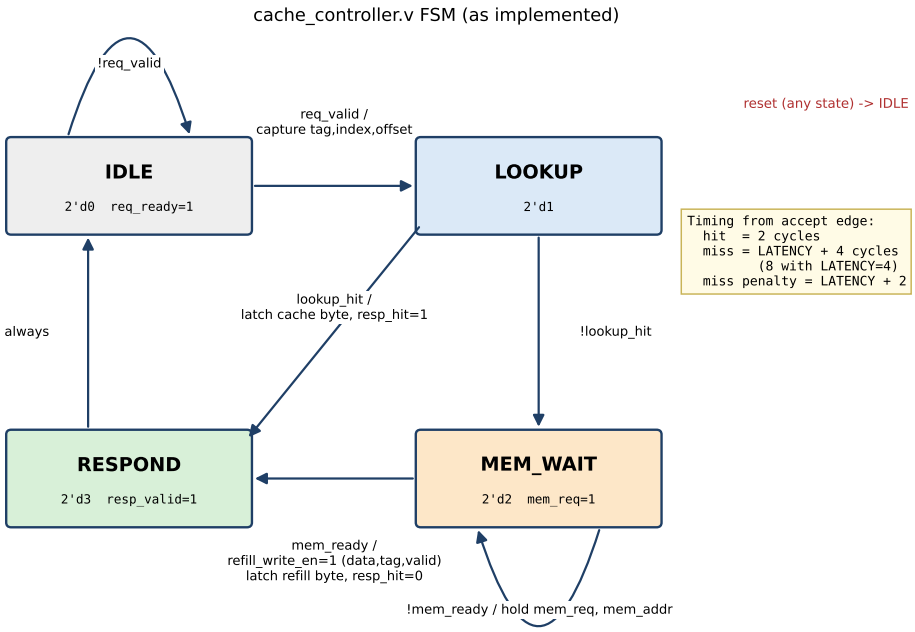



Fig. 5. Implemented controller states and transitions.

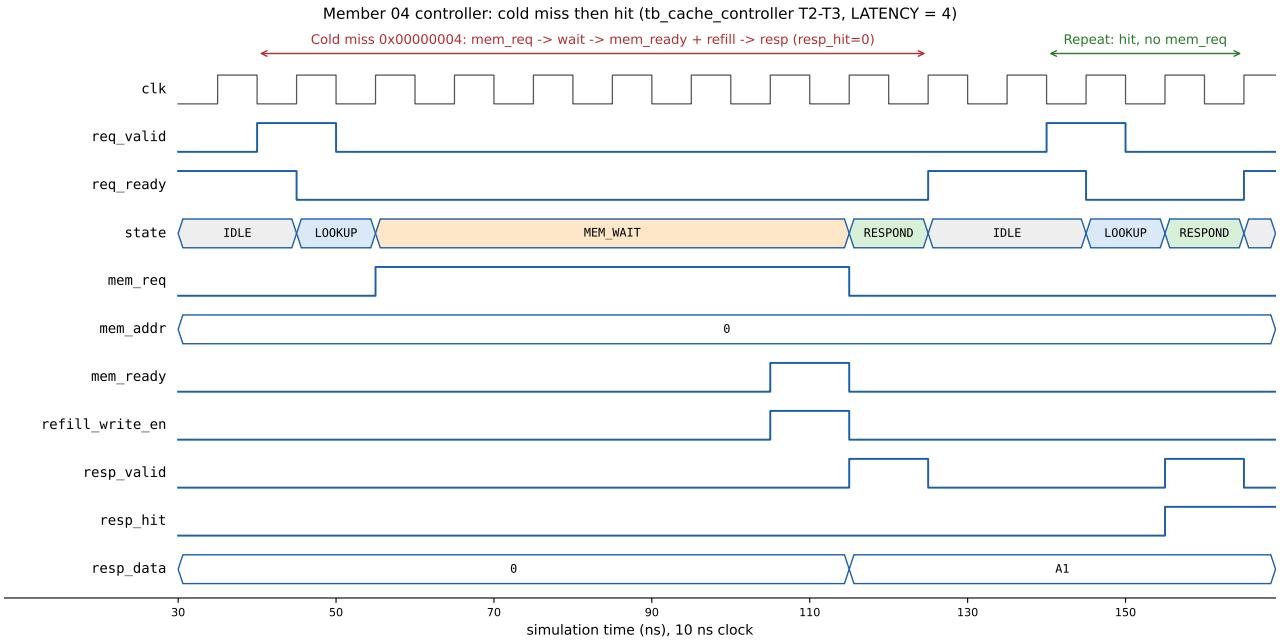



Fig. 6. Controller regression VCD showing the T2 cold miss and T3 repeat hit at byte address 0x04.

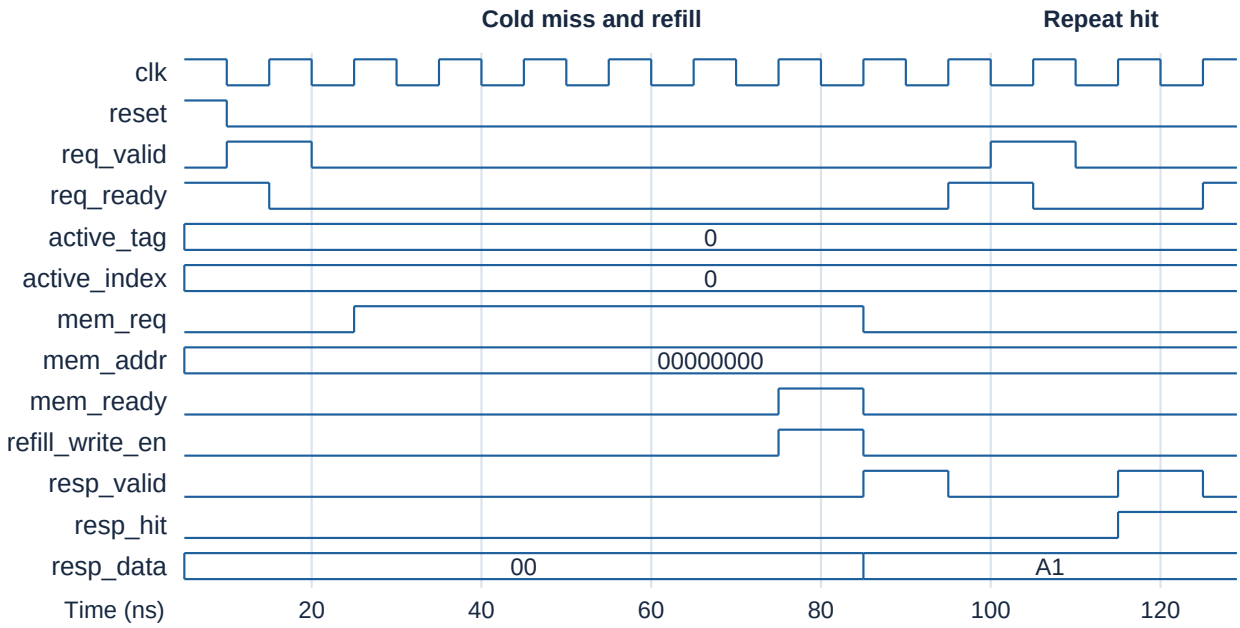



Fig. 7. Repeated workload VCD. Memory-ready qualifies the cold refill. The next read returns A1 with the hit flag high and memory idle.

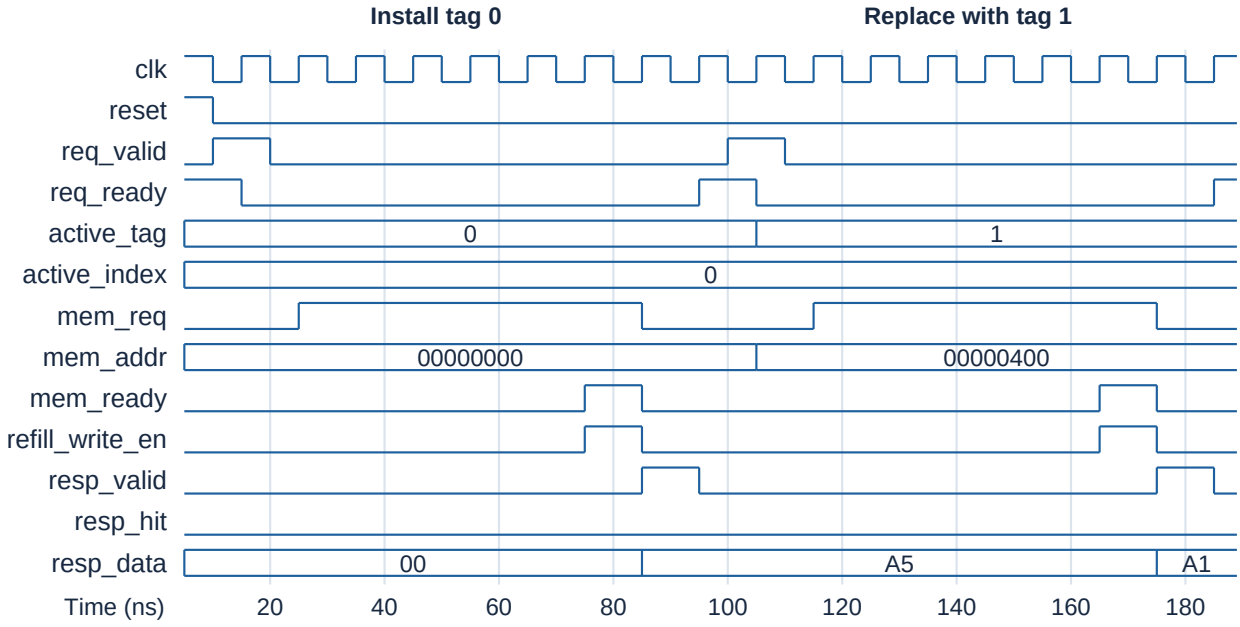



Fig. 8. Conflict workload VCD. Both requests select index zero. The second refill replaces tag zero with tag one. Buses are hexadecimal.